

# Capítulo 3: Convenciones en Python

Estándares de código, PEP 8, legibilidad industrial, estandarización de equipos y mantenibilidad en sistemas de software reales.

**Curso:** Curso Profesional de Python

**Unidad:** Capítulo 3

**Nivel:** Desde Principiante a Práctico

**Enfoque:** Desarrollo de Software & Código Limpio

## 1. CONECTANDO CONOCIMIENTOS

En el **Capítulo 1** aprendiste sobre los tipos de datos fundamentales (enteros, flotantes, cadenas) y en el **Capítulo 2** dominaste la creación y asignación de variables en memoria. Sin embargo, saber cómo funciona una sintaxis no es suficiente para ser un desarrollador profesional. El siguiente paso lógico es aprender a escribir ese código de manera estandarizada, limpia y profesional para que pueda ser leído, auditado y mantenido por cualquier desarrollador en el mundo.

## 2. OBJETIVO DEL CAPÍTULO

Este capítulo enseña las convenciones oficiales y comunitarias que rigen el ecosistema Python, enfocándose en la propuesta de mejora **PEP 8**. Aprenderás las reglas de nombramiento, espaciado, estructuración e indentación exigidas en entornos industriales de software.

### Al finalizar este capítulo serás capaz de:

- Diferenciar y aplicar correctamente los estilos de nombramiento: `snake_case`, `PascalCase`, `UPPER_CASE` y guiones bajos explicativos.
- Aplicar los estándares oficiales de formato de código según la guía PEP 8 (indentación, longitud de línea, espacios en blanco).
- Escribir documentación profesional utilizando Type Hints e historias completas mediante Docstrings.
- Configurar y utilizar herramientas automáticas de análisis de código (Linters) como `flake8`, `black` y `pylint`.
- Refactorizar código desordenado o heredado (legacy) transformándolo en código idiomático (Pythonic) y legible.

## 3. INTRODUCCIÓN: ¿QUÉ SON LAS CONVENCIONES?

Imagina una red vial internacional. Todos los países acuerdan que la luz roja del semáforo significa "detenerse" y la verde "avanzar". Nadie te arrestará mecánicamente si inventas tu propio color azul para detenerte, pero crearás un caos masivo y accidentes fatales en la vía. En programación, las **convenciones** son exactamente esos acuerdos comunitarios.

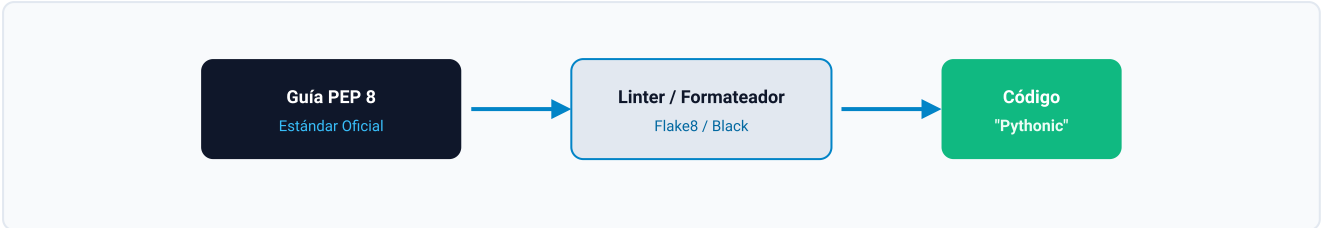
**Definición:** Una convención es un conjunto de normas y recomendaciones de estilo aceptadas por la comunidad de desarrollo que definen cómo debe escribirse el código fuente sin alterar su ejecución lógica.

## 4. ¿POR QUÉ EXISTEN Y QUÉ PROBLEMA RESUELVEN?

El código fuente se escribe una sola vez, pero se lee, revisa y modifica cientos de veces durante su ciclo de vida. Según los creadores de Python, "la legibilidad cuenta".

**Sin Convenciones (Código Legacy Inmantenible)**  
  
Nombres crípticos (x1, temp), mezcla de estilos (CamelCase con snake\_case), falta de espacios, código denso sin sangría uniforme. Resulta en horas perdidas descifrando la intención original.

**Con Convenciones PEP 8 (Código Pythonic)**  
  
Estructura limpia, autodocumentada, nombramiento intuitivo. Cualquier programador puede integrarse al proyecto e interpretar el funcionamiento inmediatamente.



## 5. ESTILOS DE NOMBRAMIENTO (NAMING CONVENTIONS)

Dependiendo del tipo de elemento de software que estés creando en Python (variable, constante, función, clase o módulo), se debe utilizar una convención tipográfica específica.

| Elemento              | Estilo Requerido | Sintaxis                            | Ejemplo Correcto             |
|-----------------------|------------------|-------------------------------------|------------------------------|
| Variables y Funciones | snake_case       | Minúsculas separadas por guion bajo | press_psi, calcular_flujo()  |
| Constantes            | UPPER_CASE       | Mayúsculas separadas por guion bajo | LIMITE_TEMPERATURA_MAX       |
| Clases                | PascalCase       | Cada palabra inicia con mayúscula   | ControladorSensorPLC         |
| Módulos y Paquetes    | snake_case corto | Minúsculas sin guiones si es corto  | telemetria.py, robot_control |
| Variables Privadas    | _snake_case      | Inicia con un guion bajo simple     | _conexion_db                 |

**Término Clave: ¿Qué es "Pythonic"?**  
  
Es un adjetivo utilizado por la comunidad para describir código que no solo es sintácticamente correcto, sino que utiliza las características, modismos y convenciones de estilo propias de Python de la forma más elegante e intuitiva posible.

## 6. REGLAS FORMALES DE FORMATO PEP 8

---

### A. Indentación (Sangría)

Python no utiliza llaves {} para delimitar bloques de código. La estructura lógica depende 100% de la sangría. PEP 8 establece estrictamente el uso de **4 espacios en blanco por nivel de anidación**. Queda totalmente prohibido el uso de Tabuladores (Tab) o mezclar ambos.

### B. Longitud de Línea y Espaciado

- **Longitud máxima:** Las líneas no deben exceder los **79 caracteres** para permitir abrir múltiples archivos lado a lado en un monitor profesional.
- **Espacios alrededor de operadores:** Usa siempre un espacio antes y después de los operadores de asignación o matemáticos (=, +, -, ==). *Excepción:* No uses espacios al definir parámetros por defecto o anotaciones en funciones.
- **Líneas en blanco:** Separa las funciones de nivel superior y las clases con **2 líneas en blanco**. Dentro de métodos o funciones, usa 1 línea en blanco para separar secciones lógicas.

```
# CORRECTO (PEP 8 Cumplido) def calcular_rendimiento_motor(velocidad_rpm, presion_bar):  
    factor_eficiencia = 0.85  
    resultado = (velocidad_rpm * presion_bar) * factor_eficiencia  
    return resultado  
def verificar_alarma(temperatura):  
    if temperatura > 90.0:  
        return True  
    return False
```

```
# INCORRECTO (Viola PEP 8) def calcularRendimientoMotor(velocidad_rpm,presion_bar):  
    # Falta espacio tras coma y nombre en CamelCase  
    factor=0.85  
    # Indentacion de 2 espacios en lugar de 4  
    resultado=(velocidad_rpm*presion_bar)*factor  
    # Sin espacios en operadores  
    return resultado
```

## 7. TYPE HINTS Y DOCSTRINGS (ESTANDARIZACIÓN AVANZADA)

---

En el software profesional, indicar de forma explícita los tipos de datos que espera una función y documentar su comportamiento es una convención mandatoria.

```
# Uso de Type Hints (Anotaciones de Tipo) y Docstrings estilo Google  
def procesar_lectura_sensor(id_sensor: str, valor_voltaje: float) -> bool:  
    """Procesa y valida la lectura analógica de un sensor industrial.  
    Args: id_sensor (str): Identificador único del PLC o sensor.  
    valor_voltaje (float): Voltaje medido en la entrada analógica (0.0 a 10.0 V).  
    Returns: bool: True si la lectura está dentro de rangos operativos,  
    False de lo contrario. """  
    VOLTAJE_MAXIMO: float = 9.5  
    return valor_voltaje <= VOLTAJE_MAXIMO
```

## 8. EJEMPLOS PROGRESIVOS POR NIVEL

---

### Nivel 1: Ejemplo Básico (Nombramiento y Constantes)

```
# Definición estandarizada de constantes y variables básicas
PIEZA_CODIGO_BASE = "PN-990-B"
STOCK_MINIMO_PERMITIDO = 50
inventario_actual_piezas = 120
es_stock_critico = inventario_actual_piezas < STOCK_MINIMO_PERMITIDO
print("Código de Componente:", PIEZA_CODIGO_BASE)
print("¿Requiere Reabastecimiento?:", es_stock_critico)
```

**Explicación:** Las constantes están escritas en UPPER\_CASE para notificar a otros desarrolladores que su valor no debe cambiar durante la ejecución. Las variables usan snake\_case descriptivo.

### Nivel 2: Ejemplo Intermedio (Uso de Funciones con Type Hints)

```
def convertir_celsius_a_fahrenheit(temperatura_celsius: float) -> float:
    FACTOR_CONVERSION: float = 1.8
    OFFSET_FAHRENHEIT: float = 32.0
    temperatura_fahrenheit = (temperatura_celsius * FACTOR_CONVERSION) + OFFSET_FAHRENHEIT
    return temperatura_fahrenheit
temp_lectura_c = 25.5
temp_convertida = convertir_celsius_a_fahrenheit(temp_lectura_c)
print(temp_convertida) # Muestra: 77.9
```

### Nivel 3: Ejemplo Práctico (Modulo de Control de Robot Industrial)

```
# Estándar profesional de una clase de control de ejes
class ControladorBrazoRobotico:
    VELOCIDAD_MAXIMA_MM_S: int = 1500
    def __init__(self, id_robot: str):
        self.id_robot = id_robot
        self._posicion_eje_x: float = 0.0
        # Atributo protegido/privado por convención
    def mover_eje_x(self, nueva_posicion: float) -> bool:
        """Mueve el eje X verificando los límites del área de trabajo."""
        if nueva_posicion >= 0.0:
            self._posicion_eje_x = nueva_posicion
            return True
        return False
```

## 9. ERRORES DE CONFUSIÓN COMUNES

| Concepto / Error                      | Qué está haciendo mal                                      | Consecuencia Real                               | Solución Correcta                                                         |
|---------------------------------------|------------------------------------------------------------|-------------------------------------------------|---------------------------------------------------------------------------|
| <b>Mezclar Tabuladores y Espacios</b> | Pulsar Tab en unas líneas y 4 barra espaciadoras en otras. | IndentationError intermitente en ejecución.     | Configurar el editor para que convierta automáticamente Tab a 4 espacios. |
| <b>Uso de CamelCase en Variables</b>  | Escribir nivelTanqueAgua = 50 al estilo de Java/C#.        | Rompe la consistencia de proyectos Pythonic.    | Usar snake_case: nivel_tanque_agua = 50.                                  |
| <b>Nombres de una Sola Letra</b>      | Nombrar variables como d, x, temp.                         | Código críptico, imposible de mantener.         | Usar nombres explícitos: distancia_metros, temperatura_horno.             |
| <b>Sombrear built-ins</b>             | Crear una variable llamada str = "hola" o list = [1, 2].   | Invalida la función integrada nativa de Python. | Cambiar el nombre: cadena_texto, lista_datos.                             |

## 10. HERRAMIENTAS AUTOMÁTICAS (LINTERS Y FORMATEADORES)

En el software profesional moderno, los desarrolladores no verifican manualmente cada espacio o línea. Se utilizan herramientas automáticas integradas en el pipeline de desarrollo.

### Linters (Analizadores Estáticos)

Herramientas como **Flake8** y **Pylint** analizan el código sin ejecutarlo, señalando líneas exactas donde se violan las reglas de PEP 8.

### Formateadores Automáticos

Herramientas como **Black** o **yapf** reestructuran e indentan el código fuente automáticamente al guardar el archivo según estándares rigurosos.

### Comandos Utilitarios en Consola

```
# Instalar y ejecutar verificador Flake8 pip install flake8 flake8 mi_script_industria
l.py # Formatear automáticamente todo el código según PEP 8 con Black pip install black black
mi_script_industrial.py
```

## 11. EJERCICIOS PRÁCTICOS DE APLICACIÓN

### Ejercicio 1 — Comprensión Básica: Identificación de Estilos

Clasifica los siguientes nombres de identificadores según su convención (snake\_case, PascalCase, UPPER\_CASE, o Inválido/No Recomendado):

1. CalculadoraCaudal
2. velocidad\_promedio\_kmh
3. PRESS\_MAX\_VALVE
4. temperaturaTanque
5. \_llave\_acceso\_api

### Ejercicio 2 — Auditoría de Código (Encuentra las Violaciones PEP 8)

Examina el siguiente fragmento de código y enumera al menos 5 violaciones a las convenciones institucionales de Python:

```
def procesoDatos(X,Y): VALOR1=X*10.5 resultado_Final=VALOR1+Y return resultado_Final
```

### Ejercicio 3 — Integración de Type Hints y Docstrings

Escribe la definición de una función llamada `calcular_consumo_electrico` que tome el voltaje (flotante) y la corriente (flotante) y retorne la potencia\_watts (flotante). Debe cumplir con:

- Type hints en parámetros y valor de retorno.
- Docstring explícito que describa la función, argumentos y valor retornado.
- Uso estricto de snake\_case e indentación de 4 espacios.

## 12. MINI PROYECTO DEL CAPÍTULO

---

### Proyecto: Refactorización Profesional de Sistema Legacy de Telemetría

**Contexto:** Un cliente industrial te entrega un script obsoleto y desordenado que lee datos de una caldera de condensación. El script funciona, pero nadie en la planta comprende el código y los linters generan decenas de advertencias. Tu objetivo es refactorizarlo al 100% bajo los estándares PEP 8.

### Código Legacy Recibido (A Refactorizar):

```
# SCRIPT ORIGINAL DESORDENADO (NO TOCAR DIRECTAMENTE, USAR COMO BASE) TempMax = 120.0 def CheckCaldera(t,p): if t>TempMax: r="ALERTA CRITICA" else: r="OPERACION NORMAL" return r print(CheckCaldera(135.5,2.1))
```

### Requisitos Técnicos de Refactorización:

1. Convertir la constante TempMax al estándar de constantes UPPER\_CASE con un nombre descriptivo (ej. TEMPERATURA\_MAXIMA\_CALDERA\_C).
2. Renombrar la función CheckCaldera a snake\_case en español o inglés consistente.
3. Renombrar los parámetros ambiguos t y p a nombres completamente explicativos con Type Hints.
4. Agregar un Docstring descriptivo completo a la función.
5. Corregir la indentación a 4 espacios exactos y espaciado alrededor de operadores lógicos.

## 13. COMPRUEBA TUS CONOCIMIENTOS (AUTOEVALUACIÓN)

---

1. ¿Cuál es el documento oficial de la propuesta de mejora de Python que define las convenciones de estilo de código?
2. ¿Cuántos espacios en blanco deben utilizarse por cada nivel de sangría según PEP 8?
3. ¿Cuál es la diferencia entre la convención de nombramiento para una Variable (`snake_case`) y una Clase (`PascalCase`)?
4. ¿Por qué se considera una mala práctica nombrar una variable como `list = [10, 20, 30]`?
5. ¿Qué herramienta estática puedes usar en la terminal para detectar errores de formato en tus archivos sin ejecutarlos?

## RESPUESTAS Y EXPLICACIONES

---

1. **Respuesta 1:** El documento oficial es la **PEP 8** (Python Enhancement Proposal #8).
2. **Respuesta 2:** Deben utilizarse exactamente **4** espacios en blanco. Nunca utilizar tabuladores.
3. **Respuesta 3:** `snake_case` utiliza minúsculas separadas por guion bajo (ej. `control_flujo`), mientras que `PascalCase` inicia cada palabra con mayúscula sin separadores (ej. `ControlFlujo`).
4. **Respuesta 4:** Porque `list` es una palabra reservada/función nativa de Python. Al usarla como variable, sobrescribes la función nativa impidiendo crear listas mediante `list()`.
5. **Respuesta 5:** Herramientas como **Flake8** o **Pylint**.

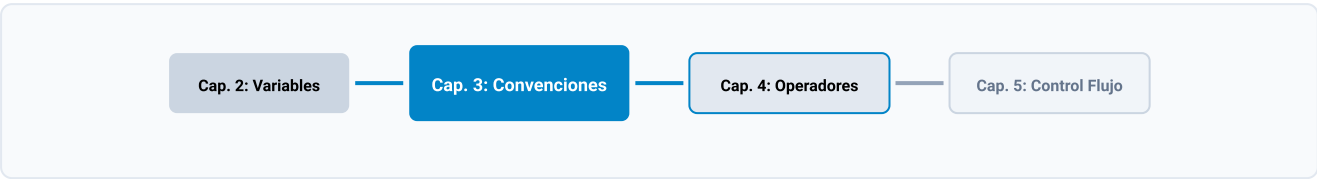
## 14. RESUMEN DEL CAPÍTULO

| Elemento   | Estilo Estándar | Ejemplo Pythonic  | Regla Clave                                    |
|------------|-----------------|-------------------|------------------------------------------------|
| Variables  | snake_case      | presion_psi       | Usar sustantivos o adjetivos en minúscula.     |
| Constantes | UPPER_CASE      | VELOCIDAD_LUZ     | Valores fijos que no cambian en ejecución.     |
| Funciones  | snake_case      | calcular_area()   | Utilizar verbos que expresen acción.           |
| Clases     | PascalCase      | SensorTemperatura | Sustantivos en singular con mayúscula inicial. |

### Ideas Clave que Debes Recordar

- El código se lee mucho más de lo que se escribe; prioriza siempre la legibilidad.
- Usa siempre **4 espacios por nivel de indentación**.
- Las constantes se escriben en MAYÚSCULAS\_CON\_GUIÓN\_BAJO.
- Usa Flake8 y Black para automatizar la calidad y el formato del código en tus proyectos.

## 15. MAPA DE APRENDIZAJE PROMOCIONAL



**Siguiente Paso:** Con las variables dominadas y el estándar de código limpio integrado en tus hábitos, en el **Capítulo 4: Operadores y Expresiones** aprenderás a realizar operaciones matemáticas, lógicas y de comparación con una precisión impecable.